

Trabajo Practico N7

Cátedra: Ingeniería en Software II

Docente: Dr. Pedro Colla - Lic. Lucia Blanc

Alumnos: Juan Balbuena - Nahuel Zaragoza - Romero Gonzalo - Romero

Valentin - Sangalli Luciano Repositorio

utilizado: https://github.com/NahuelZarago/IS2_Tp1/tree/tpsoftware/P7_IS2

1)

a y b)

```
17  class JSONTokenReader:
18      """
19      Clase que implementa el patrón Singleton para gestionar la lectura
20      centralizada y única de la configuración desde un archivo JSON.
21      """
22      _instance = None
23      _data = None
24
25      def __new__(cls, *args, **kwargs):
26          """
27          Garantiza que exista una única instancia de la clase en memoria.
28          """
29          if cls._instance is None:
30              cls._instance = super(JSONTokenReader, cls).__new__(cls)
31          return cls._instance
32
33      def __init__(self, filename="sitedata.json"):
34          """
35          Inicializa la instancia cargando los datos si aún no han sido procesados.
36          """
37          # Evita recargar el archivo si la instancia ya fue inicializada previamente
38          if self._data is None:
39              self._filename = filename
40              self._load_data()
41
```

c)

```
def _load_data(self):
    """
    Carga y parsea el archivo JSON de forma segura. Controla los errores
    para que el sistema nunca lance una excepción cruda del lenguaje.
    """
    try:
        with open(self._filename, "r", encoding="utf-8") as myfile:
            content = myfile.read()
            self._data = json.loads(content)
    except FileNotFoundError:
        print(f"Error del programa: No se pudo encontrar el archivo "
              f"obligatorio '{self._filename}'.")
        sys.exit(1)
    except json.JSONDecodeError:
        print(f"Error del programa: El archivo '{self._filename}' "
              f"no contiene un formato JSON válido.")
        sys.exit(1)
    except PermissionError:
        print(f"Error del programa: Permisos insuficientes para leer "
              f"'{self._filename}'.")
        sys.exit(1)
```

d)

```
def parse_arguments():
    """
    Valida y procesa los argumentos de la línea de comandos de forma robusta.
    Evita la terminación abrupta por excepciones del lenguaje.
    """
    # Si hay más de un parámetro, el uso es incorrecto
    if len(sys.argv) > 2:
        print("Error del programa: Demasiados argumentos. ")
        print("Uso: python getJason_mod.py [clave | -v]")
        sys.exit(1)

    # Si no se pasan argumentos, se usa el token por defecto
    if len(sys.argv) == 1:
        return "token1"

    argument = sys.argv[1]

    # Requerimiento g): Control de flag de versión
    if argument == "-v":
        print("versión 1.1")
        sys.exit(0)

    # Control de argumentos inválidos (ej. flags vacíos o extraños si se requiriera)
    if not argument.strip():
        print("Error del programa: El argumento provisto no puede estar vacío.")
        sys.exit(1)

    return argument
```

e y f)

```

def parse_arguments():
    """
    Valida y procesa los argumentos de la línea de comandos de forma robusta.
    Evita la terminación abrupta por excepciones del lenguaje.
    """
    # Si hay más de un parámetro, el uso es incorrecto
    if len(sys.argv) > 2:
        print("Error del programa: Demasiados argumentos. "
              "Uso: python getJason_mod.py [clave | -v]")
        sys.exit(1)

    # Si no se pasan argumentos, se usa el token por defecto
    if len(sys.argv) == 1:
        return "token1"

    argument = sys.argv[1]

    # Requerimiento g): Control de flag de versión
    if argument == "-v":
        print("versión 1.1")
        sys.exit(0)

    # Control de argumentos inválidos (ej. flags vacíos o extraños si se requiriera)
    if not argument.strip():
        print("Error del programa: El argumento provisto no puede estar vacío.")
        sys.exit(1)

    return argument

```

g)

```

def parse_arguments():
    """
    Valida y procesa los argumentos de la línea de comandos de forma robusta.
    Evita la terminación abrupta por excepciones del lenguaje.
    """
    # Si hay más de un parámetro, el uso es incorrecto
    if len(sys.argv) > 2:
        print("Error del programa: Demasiados argumentos. "
              "Uso: python getJason_mod.py [clave | -v]")
        sys.exit(1)

    # Si no se pasan argumentos, se usa el token por defecto
    if len(sys.argv) == 1:
        return "token1"

    argument = sys.argv[1]

    # Requerimiento g): Control de flag de versión
    if argument == "-v":
        print("versión 1.1")
        sys.exit(0)

    # Control de argumentos inválidos (ej. flags vacíos o extraños si se requiriera)
    if not argument.strip():
        print("Error del programa: El argumento provisto no puede estar vacío.")
        sys.exit(1)

    return argument

```

h)

```

• (venv) PS C:\Users\Luciano\Desktop\tp7> pylint getJason_mod.py
***** Module getJason_mod
getJason_mod.py:71:0: C0303: Trailing whitespace (trailing-whitespace)
getJason_mod.py:125:0: C0304: Final newline missing (missing-final-newline)
getJason_mod.py:1:0: C0103: Module name "getJason_mod" doesn't conform to snake_case naming style (invalid-name)

-----
Your code has been rated at 9.43/10

```